Phase 1: Environment Setup & Catalog Ingestion

Overview

Real-world software engineering rarely begins with perfectly clean, structured databases. Systems must often integrate with legacy formats and unstructured data. This phase bridges the gap between raw, messy data (an HTML table) and a strict, predictable backend state (JSON). You will initialize a professional development environment and build administrative API endpoints capable of securely ingesting files, parsing the Document Object Model (DOM), and exposing clean data.

Important Note on Testing Data: You have been provided with a sample `Course Catalog.html` file. This file is strictly for your local **debugging and development**. However, your final submission will be evaluated using a **hidden grading file** that has the exact same HTML structure (tables, classes, tags) but contains completely different fictitious courses and prerequisites. Your code must be generalized to parse the *structure* of the document, not hardcoded to look for specific text like "COSC 3506".

You will build your API using **Python (FastAPI)**.

> Here is a quick guide on Fast API to get you started: Python FastAPI Tutorial

---

Section 1: Version Control & Environment Initialization

What to do: Initialize a Git repository and set up an isolated project environment using Python's built-in virtual environment manager.

Why we do it:

- **The AI Safety Net:** Generative AI can quickly output hundreds of lines of code. If an AI suggestion breaks your application, Git allows you to instantly revert to a working state.
- **The "It works on my machine" Problem:** By initializing a virtual environment, you ensure that when the automated grader runs your code, it installs the exact same library versions you used locally.

Step-by-Step CLI Instructions:

```
mkdir course-registration-api
cd course-registration-api

git init
git branch -M main

# Create and activate an isolated virtual environment
python -m venv venv

# Linux
source venv/bin/activate

# Windows
venv\Scripts\activate
```

Section 2: Installing Dependencies

What to do: Install a web framework to route HTTP requests, a library to handle file uploads, and an HTML parser to read the DOM.

Why we do it: Good software engineers do not reinvent the wheel. Writing a custom parser for `multipart/form-data` (binary file uploads) from scratch is dangerous and error-prone. We rely on battle-tested libraries so we can focus entirely on the business logic.

Step-by-Step CLI Instructions:

```
pip install fastapi uvicorn python-multipart beautifulsoup4
pip freeze > requirements.txt
```

Section 3: The Data Ingestion Pipeline

What to do: Build an endpoint at `POST /api/v1/admin/catalog/import` configured to accept `multipart/form-data` uploads. Use your sample `Course Catalog.html` file to test this locally. You must parse the HTML DOM, extract the Course Code, Title, Credits, Prerequisites, and Cross-listed courses, and save them to your application's memory.

Why we do it (and where AI fails):

- **Handling Binary Data:** Using `multipart/form-data` tells your server to process incoming heavy files in chunks, preventing memory overload.
- **The AI Engineering Challenge:** AI coding assistants can easily write the file upload boilerplate. However, they struggle to correctly interpret messy, human-readable strings (e.g., "*Requires COSC 1046 and either COSC 1047 or ITEC 1047*"). If you rely on basic AI output, your database will be populated with useless paragraphs instead of strict data arrays. You must actively engineer Regular Expressions (Regex) and prompt the AI strategically to sanitize this text into precise JSON arrays.

Section 4: The Verification Contract

What to do: Build an endpoint at `GET /api/v1/catalog/courses/{course_code}` that returns a `200 OK` with the perfectly structured JSON representation of the requested course.

Why we do it:

- **The API Contract:** A REST API is a promise to the frontend. No matter how messy the original HTML was, this endpoint guarantees that the data retrieved will strictly adhere to a predictable JSON schema.
- **State Verification:** This allows the auto-grader to query individual records to prove your `POST` ingestion actually altered the server's state correctly.

Section 5: Automated Evaluation & CI/CD

What to do: Ensure your logic does not rely on hardcoded values. Submit your repository for automated testing.

Why we do it:

- **Debugging vs. Grading:** In industry, Quality Assurance pipelines execute programmatic attacks against your endpoints using edge cases. Your local `Course Catalog.html` is for **debugging**. The auto-grader will upload a **hidden grading file** representing a fictitious university catalog.
- **Generalization over Memorization:** If your code was written to explicitly look for the string "COSC 3506", the auto-grader will fail you. Your parsing logic must be generalized to read the *structure* of the HTML (e.g., "find the third column in the table row"), not the specific text within it.

Additional Information

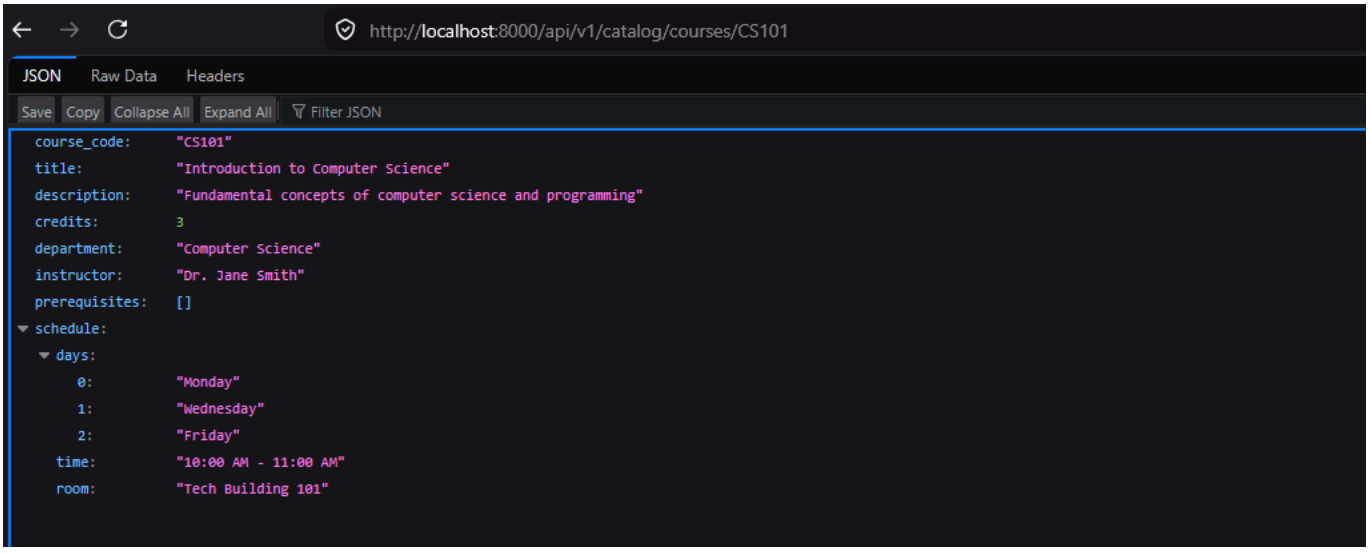
Local Testing

You might want to test your API locally before you deploy it remotely, to ensure it all works. For this, after you have coded your api, simply run it locally as shown.

```
source venv/bin/activate # or use the windows method

uvicorn main:app --reload
```

This will run the application locally, usually on port `8000`. Now, depending on how you setup your API endpoint, you should be able to acces your API as shown in the screenshot.



SSH Setup:

In order to push your code to a remote repository, you will need to configure SSH keys on your Github account and generate ssh keys on your local machine.

This handy document from github will walk you through everything you need to do to set this up: How to setup SSH Keys on GitHub

Once, you have setup your SSH keys,f follow these commands to push your changes to the remote repository.

```
git add . # Add all the changes from your folder for staging
git commit -m "Any commit Message" # Commit your changes

git remote add origin <your ssh link here>

git push -u origin main
```

> Note: You need to first create a repository on your github account and get the **SSH** URL. Make sure to avoid the HTTPS URL as it will prompt you to enter your account and password on the command prompt and most likely fail.

You might also be asked to configure your username and email, in which case the command to do so will be given in the command line itself. Simply follow the instructions and you will have uploaded your repo to github as shown below: